

Figure 1: The overall process

memory is fixed (higher memory address). Mainly, there are two operations in the collection of data, which are 'push' and 'pop'. The addition of an entity to the stack is a 'push' and the subtraction of an entity is a 'pop'. The register pointing to the top of the stack is the stack pointer (SP), which changes automatically based on the operation, and the register pointing to the bottom of the stack is the base pointer (BP). With the help of a small code snippet, we can see how the stack is framed.

```
#include<stdio.h>
int add(int i, int j);
int main(int argc, char **argv)
{
    int i;
    int j;
    int sum;
    sum= add(i,j);
    printf("Sum of two numbers = %d",sum); //assume that the
address is 0xbfff8066
    return 0;
}
int add(int i, int j)
{
    int sum;
    sum=i+j;
    return sum;
}
```

Figure 2 depicts how the stack is framed for the above problem.

The GNU debugger

Most of you might be familiar with the *printf* debugger, which can only be used if you have the source code. But for the GNU debugger, you just need the executable file to see what is happening 'inside' the program.

Here is a list of commands that are frequently used in *gdb*.

1>prompt > *gdb*.

This is to get started with *gdb*.

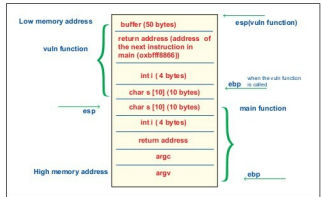


Figure 2: The stack

2> (*gdb*) file executable filename

This gives the executable file name, which you need to debug.

3> (*gdb*) run

...is to run the program in *gdb*.

4> (*gdb*) kill

...is used to kill the program being debugged.

5> (*gdb*)disass function name\

...disassembles the function into the assembler.

6> (*gdb*)b (line number) or (function name) or *(address)

...sets break points at certain points of the code.

It is very important to learn how to do this last bit because while doing exploitations, you need to set break points and analyse how the program behaves.

7> (*gdb*) x/o(octal) or x(hex) or d(decimal) or u(unsigned decimal) or t(binary) or f(float) or a(address) or i(instruction) or c(char) and s(string) (string name) or \$(register name)

This is used to examine the memory of the code.

Let's take a look at an example.

x/Is gives you what is in string 's'.

8> (*gdb*)info files or breakpoints or registers.

...will print the list of break points, files or registers.

9> (*gdb*)help command